Camera-Driven Mobile Robot Digital Twin

Physical Object Tracking and Real-Time Obstacle Synchronisation in CoppeliaSim

Stack: Python 3.10 · OpenCV 4.8 · CoppeliaSim Edu 4.10 (ZeroMQ API) · Ubuntu 22.04 LTS

System Overview

This project implements a real-time digital twin connecting a desktop camera to the **CoppeliaSim** robotics simulator. A physical camera observes a defined desk working area and tracks either a printed ArUco marker or a colored object (Red, Green, Blue). The extracted 2D pixel coordinates are mapped onto a 3.0m × 3.0m virtual arena, updating a simulated obstacle position in real time. Inside the virtual environment, a **Pioneer P3DX** mobile robot autonomously patrols a perimeter path and halts safely whenever the obstacle is moved into its forward route.

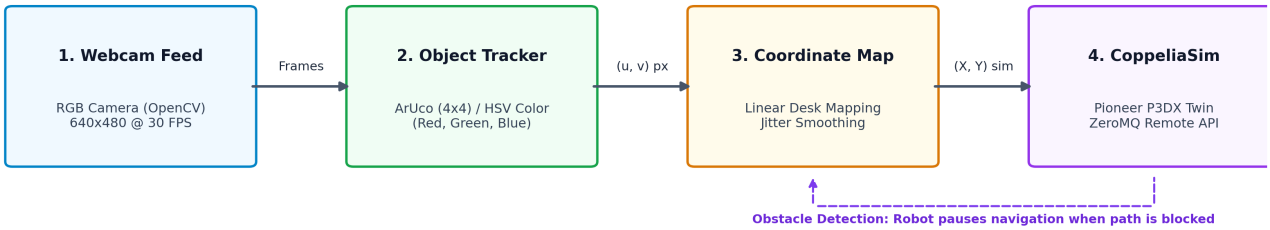


Figure 1: Digital twin pipeline: camera tracking → coordinate mapping → CoppeliaSim simulation.

The pipeline operates in real time at **30 FPS** through four connected steps:

1. **Video Capture:** Captures 640 × 480 RGB frames from a standard USB webcam using OpenCV.
2. **Object Tracking:** Extracts the physical object centroid (u, v) in image space.
3. **Coordinate Mapping:** Normalizes pixel coordinates to arena coordinates $(X_{\text{sim}}, Y_{\text{sim}})$ with jitter smoothing.
4. **Simulation Interface:** Synchronises obstacle position and reads Pioneer robot telemetry via the ZeroMQ API.

Physical Object Tracking

The vision module provides two reliable tracking modes without requiring heavy deep-learning dependencies:

- **ArUco Marker Tracking:** Detects standard `DICT_4X4_50` fiducial markers. The detector resolves marker corner coordinates and computes the center point (c_u, c_v). This provides robust, flicker-free tracking across varying room illumination.
- **HSV Color Tracking:** Implements color segmentation with calibrated presets for common colored markers:
  - **Blue Preset [Key 1]:** $H \in [100, 135]$, $S \in [100, 255]$, $V \in [60, 255]$
  - **Green Preset [Key 2]:** $H \in [35, 85]$, $S \in [70, 255]$, $V \in [50, 255]$
  - **Red Preset [Key 3]:** Handles hue wraparound ($H \in [0, 10] \cup [168, 180]$, $S \in [100, 255]$, $V \in [70, 255]$)

  Noise is cleaned using morphological opening and closing with a 5×5 kernel. The object center is calculated from spatial moments ($c_u = M_{10}/M_{00},\ c_v = M_{01}/M_{00}$).

Desk-to-Arena Coordinate Transformation

A region of interest (ROI) on the camera view defines the active desk workspace:

$$u \in [u_{\min}, u_{\max}] = [100, 540] \text{ px}, \quad v \in [v_{\min}, v_{\max}] = [60, 420] \text{ px} \tag{1}$$

Coordinates are normalized to $\bar{u}, \bar{v} \in [-1.0, 1.0]$ and scaled linearly to virtual arena coordinates $(X_{\text{sim}}, Y_{\text{sim}}) \in [-1.4\text{m}, +1.4\text{m}]$. Hand jitter during physical object movement is smoothed using an Exponential Moving Average (EMA) filter:

$$\mathbf{p}_{\text{smooth}}[t] = \alpha \cdot \mathbf{p}_{\text{raw}}[t] + (1 - \alpha) \cdot \mathbf{p}_{\text{smooth}}[t-1], \quad \alpha = 0.70 \tag{2}$$

If the object is removed from the camera view, the system maintains its last known position for 2.0 seconds before hiding the obstacle, preventing abrupt jump artifacts in the simulation.

CoppeliaSim Environment & Robot Navigation

The virtual room is modelled in **CoppeliaSim Edu 4.10** as an open 3.0m × 3.0m square box arena with 0.45m wall heights. The lowered wall height ensures full visibility of the robot, path, and obstacle from an overhead isometric perspective.

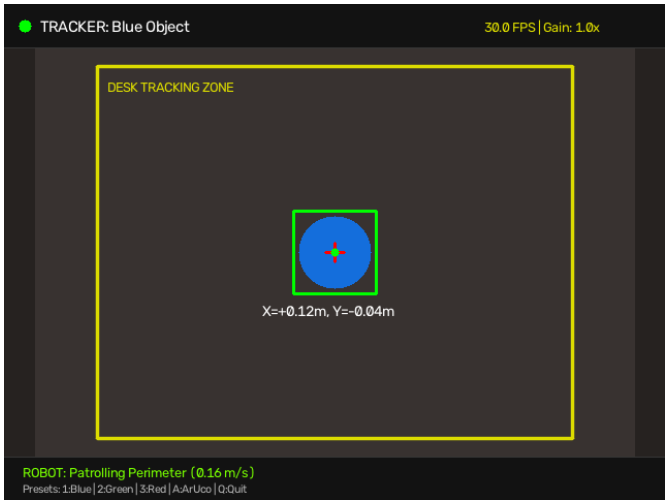


Figure 2: OpenCV tracking HUD with desk ROI.

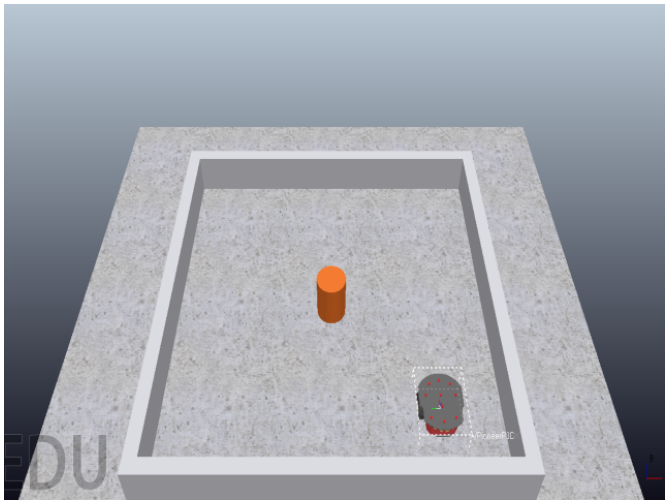


Figure 3: CoppeliaSim 3D arena with Pioneer robot.

Perimeter Waypoint Following

The differential-drive **Pioneer P3DX** mobile robot navigates a continuous square perimeter route around the arena across four corner waypoints:

$$\mathbf{W}_0(+0.95, -0.95) \longrightarrow \mathbf{W}_1(+0.95, +0.95) \longrightarrow \mathbf{W}_2(-0.95, +0.95) \longrightarrow \mathbf{W}_3(-0.95, -0.95) \tag{3}$$

- **Straight Navigation:** The robot drives toward the active waypoint at linear velocity $v = 0.16$ m/s with proportional yaw correction ($k_p = 0.85$) to keep heading aligned.
- **In-Place Corner Turning:** When the robot reaches within 0.16m of a corner waypoint, it stops linear motion ($v = 0.0$ m/s) and turns in place ($\omega = 0.60$ rad/s) until oriented toward the next waypoint. This prevents wide turns that could bring the robot into contact with boundary walls.

Obstacle Detection & Safety Halting

The robot monitors the physical obstacle's digital twin position relative to its current pose:

1. **Forward Cone Safety Check:** The controller calculates the distance and heading angle to the virtual obstacle. If the obstacle is within 0.65m and inside a forward heading cone of ±50°, wheel velocities are set to zero (`OBSTACLE BLOCKING PATH`). When the physical object is moved away, the robot resumes its perimeter route.
2. **Front Ultrasonic Clearance:** Readings from the robot's front ultrasonic sensors (indices 2 through 5) are polled in real time. If an object is detected closer than 0.18m, the robot halts as a safety safeguard.
3. **State Recovery:** If the robot tilts or leaves the floor boundary ($Z < 0.05$m), the interface automatically resets its position safely to waypoint $\mathbf{W}_0$.

System Configuration & Parameters

Component	Configuration / Value	Description
Camera Input	640 × 480 @ 30 FPS	Standard USB webcam via OpenCV V4L2 backend
Vision Tracking	ArUco (4x4) and HSV Presets	Supports ArUco marker or Red, Green, Blue objects
Desk Region of Interest	$[100, 540] \times [60, 420]$ px	Active desk area mapped to $[-1.4\text{m}, +1.4\text{m}]$ in arena
Position Smoothing	EMA filter ($\alpha = 0.70$)	Eliminates hand tremor during object positioning
Simulation Link	ZeroMQ Remote API (port 23000)	Real-time position updates and sensor telemetry
Patrol Speed	0.16 m/s cruise, 0.60 rad/s turn	Waypoint following with in-place corner turns
Obstacle Safety Stop	Distance < 0.65m, angle $< \pm 50°$	Halts robot when obstacle enters forward path
Frontal Ultrasonic Limit	Distance < 0.18m	Secondary hardware sensor clearance check

Conclusion

This system demonstrates a practical physical-to-virtual digital twin. By linking real-time computer vision with simulated mobile robotics, physical objects moved on a desk surface dynamically control virtual obstacles in CoppeliaSim, enabling the Pioneer P3DX mobile robot to interact with and respond safely to real-world physical placement.